

Master Requirements Specification

Multi-Tenant AI Voice Agent Platform Using LiveKit

1. Project Objective

Build a production-ready, multi-tenant SaaS platform for AI-powered voice agents using **LiveKit as the realtime media and SIP/telephony infrastructure**.

The platform will receive calls from external PBXs, SIP trunks, or PSTN-connected systems and route those calls through LiveKit to dynamically configured AI voice agents.

The initial architecture must support development and testing with a small number of concurrent calls, while being architected for eventual production operation at **1,000+ concurrent AI voice calls**.

The system must be horizontally scalable. Adding additional LiveKit, SIP, API, or AI worker capacity must not require application-code changes.

The platform must be **configuration-driven**. Tenant-specific behavior must never be hard-coded.

2. Core Architectural Principle

The system must be divided into two major planes:

Control Plane

Responsible for configuration, administration, authentication, authorization, tenants, AI agents, routing, provider configuration, LiveKit resource management, monitoring and business configuration.

Web UI

|

v

Configuration / Management API

|

+-----+

|

|

v

v

PostgreSQL

LiveKit APIs

|

v

Redis / Event System

Voice Execution Plane

Responsible for realtime calls and AI processing.

PBX / SIP / PSTN

|

v

LiveKit SIP

|

v

LiveKit

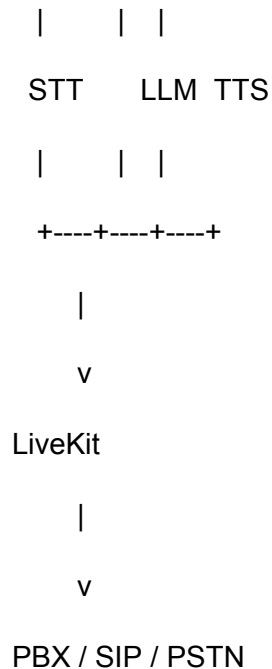
|

v

AI Agent Worker Pool

|

+---+---+---+



The Control Plane must not process the realtime audio stream.

The Voice Execution Plane must not contain tenant-specific hard-coded configuration.

3. Technology Stack

Backend

- Python
- FastAPI
- Pydantic
- SQLAlchemy
- Alembic
- AsyncIO

Frontend

Use a modern web framework such as:

- React

- Next.js
- TypeScript

The UI must be responsive and suitable for desktop administration.

Database

- PostgreSQL
- pgvector for RAG where required

Cache / Distributed State

- Redis

Realtime

- LiveKit Server
- LiveKit SIP
- LiveKit Agents
- WebRTC
- SIP
- RTP

Object Storage

Support:

- S3
- MinIO
- S3-compatible storage

Recordings and large files must not be stored directly inside PostgreSQL.

Monitoring

- Prometheus
- Grafana

Optional:

- Loki
- Tempo

- OpenTelemetry
-

4. Containerization

All services must be containerized.

Development must run using:

- Docker
- Docker Compose

Production must be designed for:

- Docker
- Kubernetes
- Helm

The initial Docker Compose environment should contain at minimum:

postgresql

redis

livekit

livekit-sip

configuration-api

ai-agent-worker

frontend

minio

prometheus

grafana

5. Production Architecture

Production must support independent horizontal scaling of:

Frontend

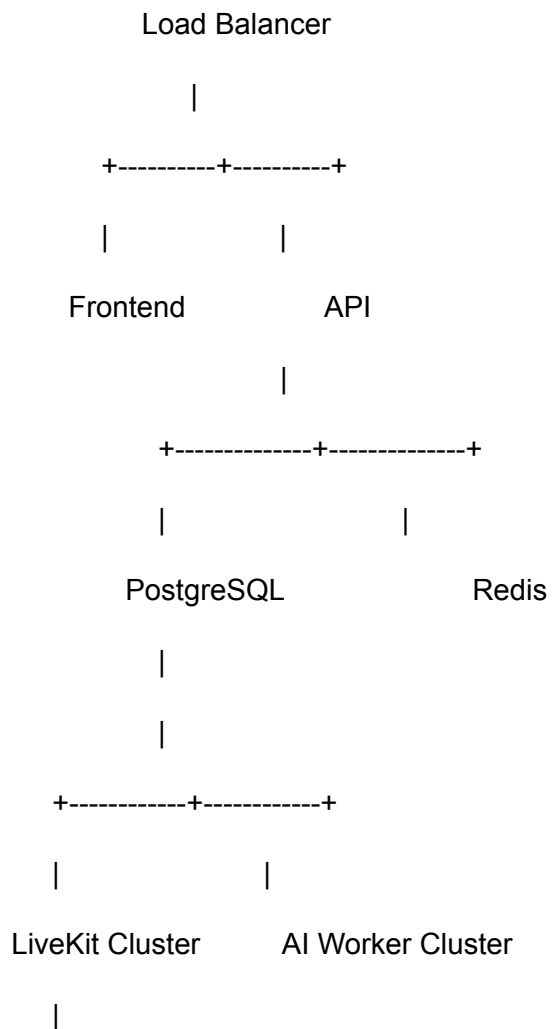
API

LiveKit

LiveKit SIP

AI Agent Workers

Conceptually:



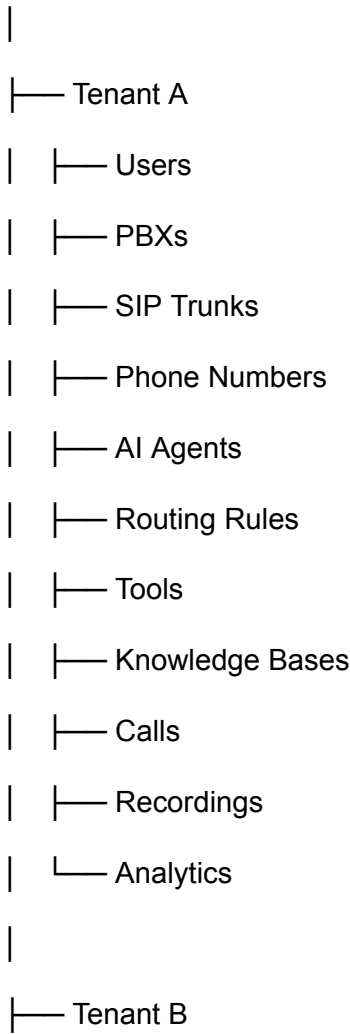
Do not design the system around one central AI process.

6. Multi-Tenancy

Multi-tenancy is mandatory from the beginning.

The platform must support:

Platform



| └─ ...
|
└─ Tenant N

Every tenant-owned database object must contain `tenant_id`.

7. Tenant Isolation

Tenant isolation must be enforced server-side.

Never trust a tenant ID supplied by the browser.

Tenant identity must come from the authenticated user's session/token.

Tenant A must never be able to access Tenant B's:

- users
- PBXs
- SIP trunks
- DIDs
- AI agents
- prompts
- provider credentials
- tools
- knowledge bases
- calls
- recordings
- transcripts
- analytics
- usage
- billing information

Tenant isolation must be tested explicitly.

8. Authentication and RBAC

Implement authentication and role-based access control.

Platform roles:

SUPER_ADMIN

PLATFORM_OPERATOR

Tenant roles:

TENANT_ADMIN

MANAGER

AGENT_MANAGER

ANALYST

VIEWER

Support granular permissions:

agents.read

agents.write

agents.publish

pbxs.read

pbxs.write

sip_trunks.read

sip_trunks.write

calls.read

recordings.read

analytics.read

users.manage

billing.read

The backend must enforce permissions independently of the frontend.

9. Configuration-Driven Architecture

No tenant-specific configuration may be hard-coded in source code.

Do not hard-code:

- tenant IDs
- PBX addresses
- SIP trunks
- DIDs
- agent mappings
- prompts
- voices
- STT providers
- LLM providers
- TTS providers
- models
- routing
- transfer destinations
- business hours
- API tools
- knowledge bases
- call limits

All must come from configuration.

10. Configuration Control Plane

Create a dedicated Configuration API.

The Configuration API is the central control plane between:

Web UI

|

v

Configuration API

|

+----- PostgreSQL

|

+----- Redis

|

+----- LiveKit APIs

|

+----- AI Worker configuration

PostgreSQL is the platform's configuration source of truth.

LiveKit is the realtime execution layer.

11. LiveKit Management

The platform must explicitly manage LiveKit.

Create an administration section:

LiveKit

- |— Clusters
- |— SIP Configuration
- |— SIP Trunks
- |— Dispatch Rules
- |— Agent Dispatch
- |— Rooms
- |— Participants
- |— Media
- |— Codecs
- |— Recording / Egress
- |— TURN / ICE
- |— Health
- └— Metrics

Where LiveKit provides an API for a resource, use the API.

Do not require administrators to manually run LiveKit CLI commands for normal tenant configuration.

12. LiveKit Resource Synchronization

When the administrator creates a LiveKit resource through the UI:

UI

|

v

Configuration API

|

v

Validate

|

v

PostgreSQL

|

v

LiveKit API

|

v

LiveKit Resource

|

v

Store LiveKit Resource ID

Example:

sip_trunks

id

tenant_id

name

livekit_trunk_id

status

Implement synchronization status:

SYNCED

PENDING

FAILED

DRIFTED

Provide:

Synchronize

Retry

Repair

13. LiveKit Infrastructure Configuration

Infrastructure-level LiveKit configuration is platform/DevOps controlled.

Examples:

- Redis
- RTC ports
- TCP ports
- RTP port ranges
- external IP
- TLS certificates
- load balancers
- Kubernetes
- networking
- firewall
- cluster topology

These must not be exposed to normal tenant administrators.

Tenant-level LiveKit configuration must be UI/API managed.

14. PBX Management

Support SIP-compatible PBXs including:

- Asterisk
- FreePBX
- FreeSWITCH
- FusionPBX
- Kamilio
- 3CX
- Cisco
- Mitel
- other SIP-compatible systems

UI:

PBXs

Name

Type

Host

Port

Transport

Status

Operations:

Create

Edit

Delete

Enable

Disable

Test Connection

15. SIP Trunk Management

Provide:

SIP Trunks

Create

Edit

Delete

Enable

Disable

Test

Configuration:

Name

PBX

Direction

SIP Host

Port

Transport

Authentication

Username

Password

Allowed IPs

Codecs

DTMF

Media Encryption

Phone Numbers

The backend must create/update the corresponding LiveKit SIP resource through the LiveKit API.

16. SIP Configuration Wizard

Provide a guided wizard:

1. Select PBX
2. SIP Configuration
3. Authentication
4. Codec
5. Security
6. Connection Test
7. Phone Number
8. AI Agent
9. Routing
10. Complete

A normal tenant administrator should not need to edit SIP configuration files.

17. Phone Number / DID Management

Create:

Phone Numbers

Fields:

Number

Tenant

PBX

SIP Trunk

Inbound Agent

Routing Rule

Business Hours

Fallback

Status

18. AI Agent Management

Create an Agent Builder.

Each agent must support:

Name

Description

Language

Greeting

System Prompt

STT Provider

STT Model

LLM Provider

LLM Model

TTS Provider

TTS Model

Voice

Temperature

Interruption

Silence Timeout

Maximum Call Duration

Recording

Transcription

Transfer Policy

Business Rules

Tools

Knowledge Base

Operations:

Create

Edit

Clone

Test

Publish

Rollback

Enable

Disable

Archive

19. Agent Versioning

Agents must have versions.

Example:

Customer Support Agent

v1 Published

v2 Draft

v3 Testing

Publishing a new version must not change existing calls.

Existing calls continue using the version with which they started.

New calls use the current published version.

20. Agent Routing

The routing engine must determine:

Incoming Call

|

v

Tenant

|

v

PBX

|

v

SIP Trunk

|

v

DID

|

v

Routing Rule

|

v

AI Agent

|

v

Published Agent Version

Routing conditions may include:

- tenant
- PBX
- SIP trunk
- DID
- caller number
- destination number
- business hours
- campaign
- priority

21. LiveKit Dispatch Rules

The system must create and manage LiveKit dispatch rules from the configuration UI/API.

Example:

Tenant:

Hotel ABC

DID:

+880XXXXXXXXXX

SIP Trunk:

Hotel PBX

Agent:

Hotel Reservation Agent

Room Strategy:

Individual Room

Agent Dispatch:

hotel-reservation-agent

Dispatch rules must be long-lived configuration objects.

Do not create a new dispatch rule for every call.

22. Per-Call Architecture

Each call should follow:

PBX

|

v

LiveKit SIP

|

v

LiveKit

|

v

SIP Dispatch Rule

|

v

Unique LiveKit Room

|

v

Agent Dispatch

|

v

AI Agent Worker

|

+---- STT

|

+---- LLM

|

+---- TTS

|

v

LiveKit

|

v

PBX

23. AI Agent Worker

The AI Agent Worker must be a generic execution engine.

It must not contain hard-coded customer/tenant logic.

At call startup it loads:

tenant

agent

agent version

prompt

STT configuration

LLM configuration

TTS configuration

voice

tools

knowledge base

transfer rules

call policies

The worker then executes that configuration.

24. AI Provider Abstraction

Implement provider interfaces.

STT

```
class STTProvider:
```

```
    async def transcribe(...)
```

LLM

```
class LLMProvider:
```

```
    async def generate(...)
```

TTS

```
class TTSPProvider:
```

```
    async def synthesize(...)
```

Provider-specific code must remain inside provider adapters.

25. Supported AI Providers

Initial provider architecture should support:

STT

- Deepgram
- OpenAI Whisper
- ElevenLabs

- Google
- Azure

LLM

- OpenAI
- Anthropic
- Google Gemini
- local/self-hosted models

TTS

- ElevenLabs
- Cartesia
- OpenAI
- Google
- Azure
- Deepgram

The architecture must allow additional providers later without redesigning the platform.

26. ElevenLabs

ElevenLabs must be supported as a TTS provider.

The implementation should support streaming TTS and integration with the LiveKit Agents voice pipeline.

Provider credentials must be configured securely.

27. Voice Library

Provide:

Voice Library

Provider

Voice ID

Voice Name

Language

Accent

Description

Status

Operations:

Add

Edit

Delete

Enable

Disable

Test

Set Default

28. Realtime Voice Pipeline

The realtime pipeline should be:

Caller

|

v

SIP

|

v

LiveKit

|

v

Audio

|

v

VAD / Turn Detection

|

v

STT

|

v

LLM

|

v

TTS

|

v

LiveKit

|

v

SIP

|

v

Caller

Use streaming wherever supported.

29. Barge-In

The agent must support natural interruption.

When the caller starts speaking while TTS is playing:

Caller Speech

|

v

Detect Interruption

|

v

Cancel TTS

|

v

STT

|

v

LLM

Support:

- VAD
 - turn detection
 - interruption
 - TTS cancellation
 - silence timeout
-

30. Tools / Function Calling

Agents must support configurable tools.

Examples:

`get_customer()`

`check_order()`

`create_order()`

`cancel_order()`

`check_inventory()`

`send_sms()`

`send_email()`

`transfer_call()`

Tools must be assignable to individual agents.

31. API Tool Builder

Provide a UI for HTTP API tools.

Fields:

Tool Name

Description

HTTP Method

URL

Authentication

Headers

Request Schema

Response Schema

Timeout

Retry Policy

Support variable substitution such as:

{{customer_id}}

{{order_id}}

{{caller_number}}

Validate tool schemas before publishing an agent.

32. Tool Permissions

Each agent must explicitly specify which tools it can use.

Example:

Sales Agent

Allowed:

- ✓ Customer API
- ✓ Inventory API
- ✓ Order API

Not Allowed:

- ✗ Refund API
- ✗ Employee API

33. Knowledge Base / RAG

Support tenant-specific knowledge bases.

Sources:

- PDF
- DOCX
- TXT
- CSV
- web content

Use PostgreSQL + pgvector initially unless another vector database is explicitly required.

Knowledge bases must be assignable to agents.

34. Conversation Memory

Each call must have isolated conversation state.

Store:

conversation_id

call_id

tenant_id

messages

tool_calls

tool_results

summary

Use summarization for long conversations.

35. Human Transfer

Support AI-to-human transfer.

Flow:

AI

|

v

Transfer Decision

|

v

Generate Conversation Summary

|

v

Select Destination

|

v

Transfer

Destinations:

SIP Extension

PBX Extension

PBX Queue

External Phone Number

SIP URI

36. Transfer Summary

Before transfer, generate:

Customer

Reason

Summary

Actions Taken

Order Information

Sentiment

Required Next Action

The summary must be configurable and available to the receiving human agent where supported.

37. Business Hours

Business hours must be configurable per tenant.

Routing must be able to use business hours.

Example:

Monday 09:00-18:00

Tuesday 09:00-18:00

Wednesday 09:00-18:00

Thursday 09:00-18:00

Friday 09:00-13:00

Saturday Closed

Sunday Closed

38. Fallback Routing

Support:

Primary AI Agent

|

X

|

Secondary AI Agent

|

X

|

PBX Queue

|

X

|

Voicemail

Fallback behavior must be configurable.

39. Call Recording

Recording must be configurable per tenant/agent.

Store recordings in object storage:

S3

MinIO

S3-compatible storage

PostgreSQL stores metadata only.

40. Transcription

Store:

Speaker

Timestamp

Text

Confidence

Speaker types:

Caller

AI

Human Agent

41. Call Database

Minimum call record:

call_id

tenant_id

agent_id

agent_version_id

pbx_id

sip_trunk_id

did

room_id

caller_number

destination_number

direction

start_time
answer_time
end_time
duration
status
hangup_reason
recording_id
transcript_id
transfer_status

42. Call State Machine

Implement:

NEW

RINGING

ANSWERED

AI_CONNECTED

IN_PROGRESS

TRANSFERRING

HUMAN_AGENT

COMPLETED

Additional states:

FAILED

TIMEOUT

CANCELLED

BUSY

NO_ANSWER

43. Call Correlation ID

Every call must have a unique `call_id`.

Propagate it through:

PBX

SIP

LiveKit

Agent

STT

LLM

TTS

Database

Logs

Recording

Transcript

This is mandatory for debugging and tracing.

44. Redis

Redis may be used for:

- active call state
- temporary state
- distributed locks
- rate limiting
- caching
- worker coordination
- configuration events

Persistent business data remains in PostgreSQL.

45. Configuration Cache

Do not query PostgreSQL for every audio/conversation event.

At call initialization:

Worker

|

v

Load configuration

|

v

Cache/local call context

|

v

Execute call

A call must use a consistent agent configuration version throughout its lifetime.

46. Configuration Drift

Detect discrepancies between PostgreSQL and LiveKit.

Example:

Database:

SIP trunk = Active

LiveKit:

SIP trunk = Missing

Show:

Configuration Drift Detected

Provide:

Synchronize

Repair

Retry

47. Tenant Call Limits

Each tenant must support:

Maximum Concurrent Calls

Maximum Daily Calls

Maximum Monthly Minutes

The platform must enforce these limits before accepting new calls.

48. Platform Capacity

SUPER_ADMIN must see:

Total Active Calls

Total Capacity

Available Capacity

LiveKit Nodes

SIP Nodes

AI Workers

Worker Utilization

CPU

Memory

Network

Provider Health

49. AI Worker Scaling

AI workers must be horizontally scalable.

Example:

Worker 1

Worker 2

Worker 3

...

Worker N

Do not assume a fixed number of workers.

The actual safe calls-per-worker must be established through load testing.

50. Kubernetes Autoscaling

Use Kubernetes HPA or an equivalent mechanism.

Scaling signals should include:

Active AI Jobs

Worker Utilization

CPU

Memory

Long-running voice calls must be treated as jobs/sessions rather than ordinary short-lived HTTP requests.

51. Graceful Shutdown

When an AI worker receives SIGTERM:

Stop accepting new calls

|

v

Finish existing calls

|

v

Disconnect cleanly

|

v

Exit

Deployments must not unnecessarily terminate active calls.

52. High Availability

Production must avoid single points of failure.

At minimum design for:

Multiple LiveKit nodes

Multiple SIP nodes

Multiple AI workers

Multiple API replicas

PostgreSQL HA strategy

Redis HA strategy

Load balancer redundancy

53. Security

Implement:

HTTPS/TLS

JWT authentication

RBAC

Tenant isolation

SIP authentication

IP allowlists

Encrypted credentials

Secret management

Audit logging

Secure object storage

54. Secret Management

Production credentials must use:

Kubernetes Secrets

or preferably:

HashiCorp Vault

AWS Secrets Manager

Google Secret Manager

Azure Key Vault

Never put production credentials in:

- source code
 - Git
 - Docker images
 - logs
 - frontend code
 - plaintext configuration
 - agent prompts
-

55. External Provider Failure

All AI providers must have:

Timeout

Retry

Exponential Backoff

Circuit Breaker

Fallback Provider

A provider failure must not crash the complete worker pool.

56. Voice Latency Monitoring

Track:

STT latency

LLM first-token latency

TTS first-audio latency

Time to first response

Time to first audio

End-to-end response latency

These metrics must be available for troubleshooting voice quality.

57. Monitoring

Use Prometheus and Grafana.

Monitor infrastructure:

CPU

Memory

Network

Disk

Containers

Pods

Restarts

LiveKit:

Rooms

Participants

Active sessions

Packet loss

Jitter

Latency

Media connections

AI:

Active calls

Agent jobs

STT latency

LLM latency

TTS latency

Errors

Worker utilization

Business:

Calls

Answered calls

Failed calls

Average duration

Transfers

AI minutes

58. Structured Logging

Use JSON logs.

Every call-related event must contain:

tenant_id

call_id

room_id

agent_id

agent_version_id

service

timestamp

event

Example:

```
{  
  "service": "voice-agent",  
  "tenant_id": "tenant_001",  
  "call_id": "call_123",  
  "agent_id": "sales",  
  "event": "tts_started"  
}
```

59. Health Checks

Every service must expose:

/health

/ready

Readiness must verify required dependencies where appropriate.

60. Platform Administration UI

Platform console:

Dashboard

Tenants

LiveKit

Infrastructure

AI Providers

Models

Voices

System Users

Monitoring

Audit Logs

Settings

61. Tenant Administration UI

Tenant console:

Dashboard

AI Agents

Agent Versions

PBXs

SIP Trunks

Phone Numbers

Routing

Business Hours

Knowledge Bases

Tools

Calls

Recordings

Transcripts

Analytics

Users

Usage

Settings

62. Agent Builder UI

A non-developer must be able to configure:

Agent Name

Greeting

System Prompt

Language

STT

LLM

TTS

Voice

Tools

Knowledge Base

Transfer

Recording

Transcription

Interruption

Timeouts

Operations:

Save Draft

Test

Publish

Rollback

63. Configuration Validation

Before publishing an agent, validate:

STT provider

LLM provider

TTS provider

Voice

Prompt

Tools

Knowledge Base

Transfer destination

Routing

PBX

SIP trunk

DID

Do not allow invalid configurations to become production-active.

64. Dependency Validation

Show configuration dependencies.

Example:

Agent

|
+-- STT

|
+-- LLM

|
+-- TTS

 |
 +-- Voice

|
+-- Tools

|
+-- Knowledge Base

|
+-- Transfer

If a required provider/voice/tool is disabled, clearly show the problem.

65. Import / Export

Allow tenant configuration export/import.

Export:

Agents

Agent Versions

Tools

Routing

Knowledge Base configuration

Business Rules

Never export secrets in plaintext.

66. Backup and Disaster Recovery

Back up:

PostgreSQL

Configuration

Recording metadata

Object storage

Redis should be treated according to whether each data type is disposable or persistent.

Provide documented disaster recovery procedures.

67. API Structure

Use:

/api/v1/

Examples:

POST /tenants

GET /tenants

POST /pbxs

GET /pbxs

PUT /pbxs/{id}

POST /sip-trunks

GET /sip-trunks

PUT /sip-trunks/{id}

POST /phone-numbers

GET /phone-numbers

POST /agents

GET /agents

PUT /agents/{id}

POST /agents/{id}/publish

POST /agents/{id}/rollback

POST /providers

GET /providers

POST /voices

GET /voices

POST /tools

GET /tools

POST /knowledge-bases

GET /knowledge-bases

POST /routing-rules

GET /routing-rules

GET /calls

GET /calls/{id}

GET /analytics

GET /usage

Generate complete OpenAPI documentation.

68. Database Schema

At minimum:

tenants

users

roles

permissions

user_roles

pbxs

sip_trunks

sip_credentials

phone_numbers

agents

agent_versions

agent_tools

agent_voices

providers

provider_credentials

models

voices

routing_rules

business_hours

transfer_destinations

knowledge_bases

knowledge_documents

tools

tool_permissions

calls

call_events

call_transcripts

call_transcript_segments

call_recordings

usage

billing

subscriptions

audit_logs

Every tenant-owned table must support tenant isolation.

69. Audit Logs

Record:

User

Tenant

Timestamp

Action

Resource

Resource ID

Old Value

New Value

IP

Examples:

agent.created

agent.updated

agent.published

pbx.created

trunk.updated

routing.changed

provider.updated

credential.rotated

70. Testing

Implement:

Unit tests

- tenant isolation
- authorization
- routing
- configuration
- agent logic
- provider adapters
- tools

Integration tests

PBX

→ SIP

→ LiveKit

→ AI Agent

→ STT

→ LLM

→ TTS

End-to-end tests

Perform actual SIP/phone calls through the complete system.

71. Load Testing

The target is 1,000 concurrent calls.

Test progressively:

10

25

50

100

250

500

750

1000

Measure:

CPU

RAM

network

SIP capacity

RTP performance

packet loss

jitter

STT latency

LLM latency

TTS latency

first-audio latency

call failure rate

worker utilization

LiveKit utilization

72. Sustained Load Testing

At high concurrency test:

15 minutes

30 minutes

60 minutes

Do not validate scalability using only short bursts.

73. Failure Testing

Test:

AI worker failure

LiveKit node failure

SIP node failure

API restart

Redis failure

Database failure

STT failure

LLM timeout

TTS failure

Network packet loss

High CPU

Worker exhaustion

Document recovery behavior.

74. Capacity Report

Claude must produce a capacity report based on actual benchmarks.

For example, if testing determines:

One worker safely handles 15 concurrent calls

then:

$1000 / 15 = 67$ workers

Add production headroom.

The numbers above are examples only.

The final worker count must come from actual benchmarking.

75. No Fake Scalability

The system must not claim:

"Supports 1,000 concurrent calls"

simply because Kubernetes can start enough containers.

The 1,000-call target must be validated using the actual production architecture and AI providers.

The load test must include:

SIP

LiveKit

AI workers

STT

LLM

TTS

recording

tools

RAG

where those features are enabled in the production configuration.

76. Development Phases

Phase 1: Basic Call

PBX

→ LiveKit SIP

→ LiveKit

→ AI Agent

→ PBX

Goal:

One complete successful call.

Phase 2: Realtime Quality

Test:

10 concurrent calls

Validate:

- audio
 - latency
 - interruption
 - STT
 - LLM
 - TTS
 - recording
 - transcript
-

Phase 3: Multi-Tenancy

Implement:

Tenants

Users

RBAC

Tenant isolation

Phase 4: Configuration Platform

Implement:

PBX UI

SIP Trunk UI

DID UI

Agent Builder

Routing UI

Provider UI

Voice UI

Phase 5: LiveKit Control Plane

Implement:

LiveKit SIP Trunk Management

Dispatch Rules

Agent Dispatch

LiveKit resource synchronization

Configuration drift detection

Phase 6: Advanced AI

Implement:

Tools

RAG

Knowledge Base

Conversation Memory

Human Transfer

Conversation Summary

Provider Failover

Phase 7: Production Infrastructure

Implement:

Kubernetes

HA

Autoscaling

Monitoring

Logging

Alerting

Backup

Disaster Recovery

Phase 8: Capacity Testing

Test:

100

250

500

750

1000

concurrent calls.

77. Final Acceptance Test

A tenant administrator must be able to perform the following entirely through the UI:

Create Tenant



Create User



Create PBX



Create SIP Trunk



Add DID



Create AI Agent



Select STT



Select LLM



Select TTS



Select Voice



Configure Prompt



Configure Tools



Configure Knowledge Base



Configure Transfer



Create Routing Rule



Publish Agent



Receive Call

No source-code modification should be required.

No direct database modification should be required.

No SSH should be required.

No manual LiveKit CLI configuration should be required for normal tenant setup.

78. Final Call Flow

The final production call flow must be:

INCOMING CALL

|

v

PBX / SIP

|

v

LiveKit SIP Cluster

|

v

LiveKit Cluster

|

v

Dispatch Rule

|

v

Unique LiveKit Room

|

v

Agent Dispatch

|

v

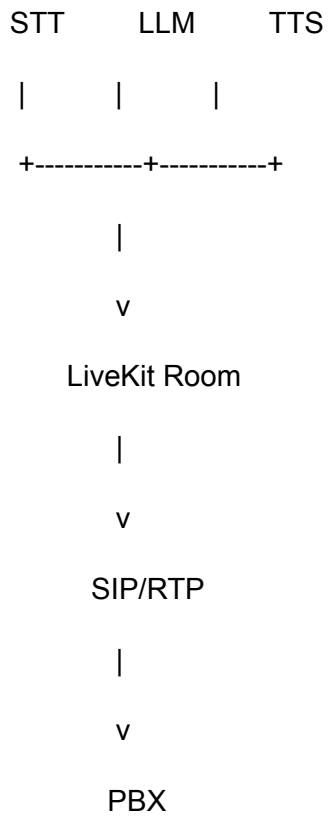
AI Worker Pool

|

+-----+-----+

| | |

v v v



79. Final Architectural Rule

The most important requirement is:

Build a generic, multi-tenant AI Voice Agent Platform, not a collection of hard-coded voice bots.

The AI Agent Worker must be a reusable execution engine.

It must dynamically obtain tenant and agent configuration from the Control Plane.

The Control Plane must manage:

Tenant

PBX

SIP Trunk

DID

Routing

AI Agent

Agent Version

STT

LLM

TTS

Voice

Tools

Knowledge Base

Transfer

Business Hours

Call Policies

Usage

PostgreSQL is the configuration source of truth.

Redis provides distributed state, caching and events.

LiveKit provides realtime media and SIP execution.

AI Agent Workers execute the configured voice-agent behavior.

The Web UI is the primary configuration interface for tenant administrators.

Infrastructure configuration remains under platform/DevOps control.

No tenant-specific configuration may be hard-coded.

The architecture must be horizontally scalable.

The 1,000 concurrent call target must be validated through actual load testing rather than assumed.

80. Claude Development Instructions

When implementing this project:

1. **Do not start by building a monolithic application.**
2. Separate Control Plane and Voice Execution Plane.
3. Design database schemas before implementing business logic.
4. Implement tenant isolation before tenant-facing features.
5. Implement RBAC at the API layer.
6. Build provider abstractions rather than coupling the agent to one AI provider.
7. Build LiveKit integration as a dedicated service/module.
8. Store LiveKit resource IDs in PostgreSQL.
9. Implement synchronization and drift detection.
10. Do not hard-code tenant or agent configuration.
11. Do not expose secrets to the frontend.
12. Use asynchronous processing for long-running operations.
13. Make AI workers horizontally scalable.
14. Make all services containerized.
15. Make the production architecture Kubernetes-ready.
16. Add health/readiness checks from the beginning.
17. Add structured logging and correlation IDs from the beginning.
18. Write automated tests for every major module.
19. Do not claim 1,000-call capacity until load testing proves it.
20. Provide clear documentation for development, deployment, configuration and operations.

Before implementing each major component, define:

Architecture

Database Model

API Contract

Security Model

Error Handling

Tests

Scaling Considerations

The resulting project must be production-oriented, modular, testable, observable and capable of evolving from a small deployment into a 1,000+ concurrent-call multi-tenant SaaS platform.